

Advanced Programming Techniques

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

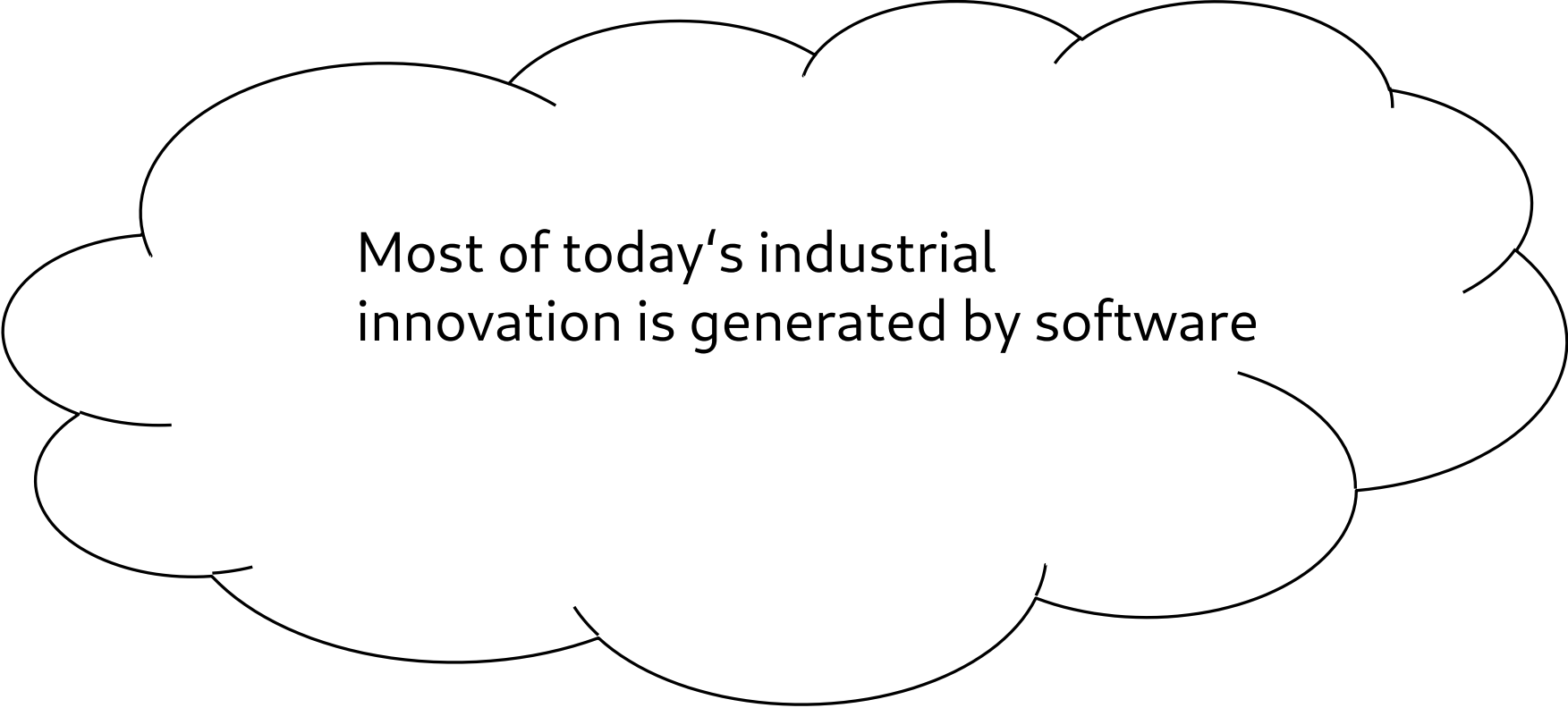
Prof. Dr. Michael Lipp

Introduction and Objects and Classes

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

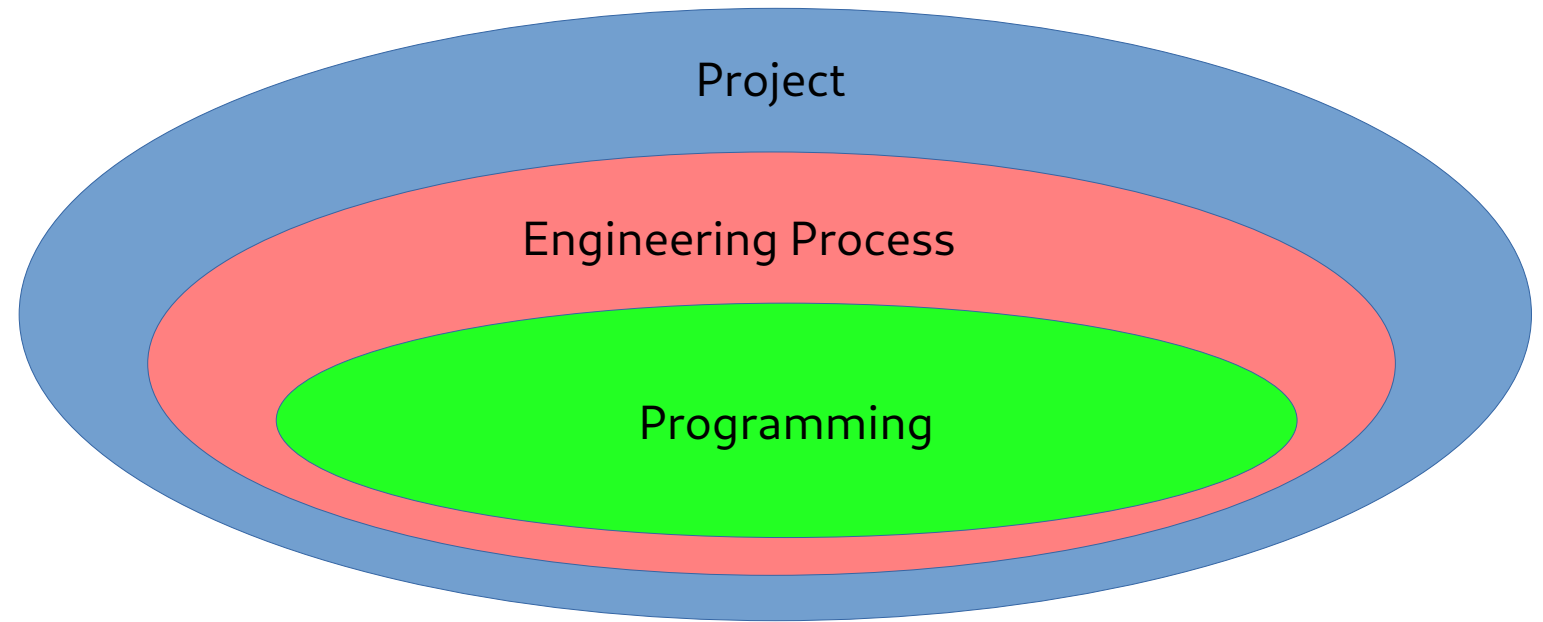
Innovation in Electrical Engineering



Most of today's industrial
innovation is generated by software

Software Development

- **Software development = programming**
 - isn't true
 - (though programming is an important part)
- **More realistic:**



- **Object orientation can dramatically increase Engineering and Programming efficiency**

C++: Object Oriented C

- **Developed by Bjarne Stroustrup**
 - Between 1979 and 1983 at the Bell Labs
 - Originally named “C with classes”
- **Added object oriented features**
 - Classes
 - Virtual functions, operator overloading
 - Multiple inheritance, templates
 - Exception handling
- **Focus on run-time performance**
- **Standardized as ISO/IEC 14882:1998, 2003 and 2011 (and more)**
- **Version from 2014 (C++14) used in lecture**



(Source: Wikimedia)

OO vs Functional Programming

- **Functional Programming**

- Code is structured in functions
- Functions (algorithms) and data are separated
- Example: `printf()`, `sin()` → Programmer has to make sure to use the correct data

- **Languages**

- C, Pascal, Fortran, Perl, ...

- **Limitations:**

- Code structure
- Protection of data
- Reuse, especially variant management

- **Benefits:**

- Very efficient code
- Easy to learn (depends)

- **Object Oriented Programming**

- Code is structured in classes
- Classes encapsulate data and functions
- Example: `cout << myComplexObject` → The object knows how it is printed

- **Languages:**

- Simula, C++, Java, C#, ...

- **Limitations:**

- Code efficiency
- Complex syntax

- **Benefits:**

- Good encapsulation
- Good reuse including variant management
- Easy to learn (if you start the OO way)

Comparing engineering approaches

Decide, which procedures you need. Then write the best possible algorithm to process your data.

Functional
programming

Decide, which modules you need. Think about the data of the modules, their abilities and their dependencies.

Object Oriented Programming

Objects and Classes

- **Objects often model things from the real or a fictional world**
 - things have state
 - things (can) expose behavior (which may change the state)
- **Classes describe**
 - how the state of a given kind of objects is represented and stored
 - which behavior a given kind of objects expose
- **Objects are *Instances of Classes***



Example: Heat Controller

(Source: <https://eurotronic.org/produkte/z-wave-heizkoerperthermostat/spirit-z-wave-plus/>)

C++ classes: “extension” of structs

simple_vector.h:

```
typedef struct {  
    int size;  
    double* elements;  
} simple_vector_t;  
  
void simple_vector_init(  
    simple_vector_t* pV,  
    int sz);  
  
void simple_vector_print(  
    simple_vector_t* pV);
```

SimpleVector.h:

```
class SimpleVector {  
private:  
    int size;  
    double* elements;  
  
public:  
    SimpleVector(int sz);  
    void print();  
};
```

C++ classes: “extension” of structs

Both are user defined types

```
typedef struct {  
    int size;  
    double* elements;  
} simple_vector_t;
```

```
void simple_vector_init(  
    simple_vector_t* pV,  
    int sz);
```

```
void simple_vector_print(  
    simple_vector_t* pV);
```

```
class SimpleVector {  
private:  
    int size;  
    double* elements;  
  
public:  
    SimpleVector(int sz);  
    void print();  
};
```

C++ classes: “extension” of structs

```
typedef struct {  
    int size;  
    double* elements;  
} simple_vector_t;
```

void simple_vector_init(
 simple_vector_t* pV,
 int sz);
void simple_vector_print(
 simple_vector_t* pV);

Class definition has
simpler Syntax

```
class SimpleVector {  
private:  
    int size;  
    double* elements;  
  
public:  
    SimpleVector(int sz);  
    void print();  
};
```

C++ classes: “extension” of structs

```
typedef struct {  
    int size;  
    double* elements;  
} simple_vector_t;
```

Data “fields” declared similarly, but are now called “data members” or “instance variables”

```
class SimpleVector {  
private:  
    int size;  
    double* elements;  
  
public:  
    SimpleVector(int sz);  
    void print();  
};
```

C++ classes: “extension” of structs

```
typedef struct {  
    int size;  
    double* elements;  
} simple_vector_t;
```

```
void simple_vector_init(  
    simple_vector_t* pV,  
    int sz);  
  
void simple_vector_print(  
    simple_vector_t* pV);
```

```
class SimpleVector {  
private:  
    int size;  
    double* elements;
```

```
public:  
    SimpleVector(int sz);  
    void print();  
};
```

Functions that take the struct as parameter become “member functions” (a.k.a. “methods”)

C++ classes: “extension” of structs

```
typedef struct {  
    int size;  
    double* elements;  
} simple_vector_t;
```

```
void simple_vector_init(  
    simple_vector_t* pV,  
    int sz);
```

```
void simple_vector_print(  
    simple_vector_t* pV);
```

```
class SimpleVector {  
private:  
    int size;  
    double* elements;
```

```
public:  
    SimpleVector(int sz);  
    void print();  
};
```

Special syntax for “init-function”,
called “Constructor”

C++ classes: “extension” of structs

```
typedef struct {  
    int size;  
    double* elements;  
};  
void simple_vector_init(  
    simple_vector_t* pV,  
    int sz);  
void simple_vector_print(  
    simple_vector_t* pV);
```

Class members (data members and member functions) have an additional visibility

```
class SimpleVector {  
private:  
    int size;  
    double* elements;  
  
public:  
    SimpleVector(int sz);  
    void print();  
};
```

C++ classes: “extension” of structs

Class definition with member function **declarations** in header file

```
class SimpleVector {  
private:  
    int size;  
    double* elements;  
  
public:  
    SimpleVector(int sz);  
    void print();  
};
```

Member function **definitions** (a.k.a. implementations) in .cpp file

```
SimpleVector::SimpleVector(  
    int sz) {  
    // ...  
}  
  
void SimpleVector::print() {  
    // ...  
}
```


C++ classes: “extension” of structs

Coding guidelines for this course:

- Class names start with upper case letter
- Member (and variable) names start with lower case letter
- All use “camel case”

```
class SimpleVector {  
private:  
    int size;  
    double* elements;  
  
public:  
    SimpleVector(int sz);  
    void print();  
};
```

C++ classes: Declaration (1)

- As with structs, classes can also be declared
 - `class x;`
- If another class is needed in header files, prefer declaration over definition if declaration is sufficient
 - Definition of the other class can be provided later in implementation file using `#include`
- (This is the same as with two functions calling each other)

C++ classes: Declaration (2)

A.h:

```
#ifndef A_H_
#define A_H_

class B;

class A {
    // Declaration is sufficient
    B* pointerToSomeB;
};

#endif
```

B.h:

```
#ifndef B_H_
#define B_H_

#include "A.h"

class B {
    // Definition of A required
    A memberOfTypeA;
};

#endif
```

Creating class source files in Eclipse

- Easily create classes using template dialogue (New/Class)

New C++ Class [x]

C++ Class

✖ Class name is empty.

Source folder: Miscellaneous/myCode [Browse...]

☐ Namespace: [Browse...]

Class name: []

Base classes:

Name	Access	Virtual
------	--------	---------

[Add...]
[Remove]
[Up]
[Down]

Uncheck!

Live Demo

- Start Eclipse
- Create class
 - Initially without constructor
 - Create Variable of type `SimpleVector` in `main.cpp`
 - Try to access member data
 - Syntax same as used for accessing fields of `structs`
 - Invoke `print()` method
 - Syntax similar to accessing fields of `structs`:
`obj.method()`

Make sure to learn how to use Eclipse features shown in lecture:

- Completion (`ctrl+sp`)
- Help on error (`ctrl+1`)
- Wizards (Source/..., e.g. Source/Implement methods) (Crucial for exam!)

Effects of visibility

- **private**

- Only accessible in member function definitions

- **public**

- Use this for member function only

- **protected**

- Related to inheritance (future topic)

Demo: data member (in)accessibility

Restricted data member visibility: Why?

- Data members should always be **private** to prohibit uncontrolled changes of state
 - Values (state) can only be changed through “setter-methods” (`void setXxx(type value)`) or as side-effect of other method invocations
 - Setter-methods can e. g. do range checks before assignment to data member to prevent illegal values
 - Member functions can ensure consistent changes (e. g. hypothetical `Vectore::resize` must consistently change size and allocated memory, simply assigning a value to `size` results in an inconsistent state).